

Coding Test – Java Backend Developer

Objective

Build a Payment Gateway Service using Spring Boot that can receive payment requests from multiple channels (Mobile Banking, Internet Banking, ATM, etc.), perform balance debit with the Core Banking System, and, if successful, forward the payment request to the Biller Aggregator.

Business Flow

1. A client/channel (Mobile Banking / Internet Banking / ATM) sends a transaction request to the Payment Gateway Service.
2. The Payment Gateway Service receives the request and:
 - Validates the transaction format and parameters.
 - Sends a debit request to the Core Banking System.
 - If the balance is insufficient → return FAILED.
 - If the balance is sufficient → proceed to the next step.
3. The Payment Gateway Service forwards the request to the Biller Aggregator to complete the payment to the merchant/biller.
4. The Biller Aggregator returns a success or failure response.
5. The Payment Gateway Service stores the transaction result in the database and sends a response back to the channel.
6. (Optional) If the transaction succeeds → publish an event to Kafka for downstream services (notification, reporting, etc.).

Note: Please include a sequence diagram or activity diagram to illustrate the flow clearly.

Requirements

1. Security

- All endpoints must be protected with JWT Authentication.
- Use Keycloak or Spring Security OAuth2 for the implementation.

2. Database Setup (PostgreSQL + JPA)

Use JPA/Hibernate as ORM for persistence and to minimize boilerplate SQL.

Create a transactions table with the following columns:

- id (UUID, Primary Key)
- order_id (String, unique)
- channel (Enum: MOBILE_BANKING, INTERNET_BANKING, ATM)
- amount (Decimal)
- account (String)
- currency (String, default: IDR)
- payment_method (String)

- status (Enum: PENDING, SUCCESS, FAILED)
- corebank_reference (String, nullable)
- biller_reference (String, nullable)
- created_at, updated_at (Timestamps, auto-generated)

3. API Endpoints

- POST /api/payments → create a new payment transaction.
- GET /api/payments/{id} → retrieve the status of a transaction.

Example Request:

```
{
  "orderId": "INV-12345",
  "channel": "MOBILE_BANKING",
  "amount": 250000,
  "currency": "IDR",
  "paymentMethod": "VIRTUAL_ACCOUNT"
}
```

Example Response (SUCCESS):

```
{
  "transactionId": "TX123456789",
  "orderId": "INV-12345",
  "status": "SUCCESS",
  "corebankReference": "CB123456789",
  "billerReference": "BILLER987654321"
}
```

Example Response (FAILED - Insufficient Balance):

```
{
  "transactionId": "TX123456789",
  "orderId": "INV-12345",
  "status": "FAILED",
  "message": "Insufficient balance"
}
```

4. Integration & Inter-Service Communication

- Implement integrations using Feign Client for synchronous REST communication.
- Support Kafka for asynchronous, event-driven inter-service communication in a microservices environment.

Core Banking API (simulate balance debit):

Endpoint: POST /api/corebank/debit

Request:

```
{ "account": "1234567890", "amount": 250000 }
```

Response:

```
{ "corebankReference": "CB123456789", "status": "SUCCESS" }
```

Biller Aggregator API (simulate biller payment):

Endpoint: POST /api/biller/pay

Request:

```
{ "orderId": "INV-12345", "amount": 250000, "paymentMethod": "VA" }
```

Response:

```
{ "billerReference": "BILLER987654321", "status": "SUCCESS" }
```

If either API call fails, the Payment Gateway must return FAILED.

5. Architecture & Best Practices

- Follow a layered architecture (Controller → Service → Repository).
- Implement structured exception handling.
- Apply centralized logging (Log4j2/SLF4J).
- Unit and integration test coverage must be at least 70%.
- Use DTOs for request/response instead of exposing entities directly.

6. Optional (Plus Points)

- Publish transaction.success events to Kafka.
- Client uses JWT token to call the Payment Service gateway
- Implement circuit breaker / retry mechanism (e.g., Resilience4j) for biller integration failures.
- Provide Swagger/OpenAPI documentation.
- Add Dockerfile + docker-compose for easy project setup.
- Use JPA/Hibernate for ORM to improve maintainability and reduce boilerplate SQL.
- Leverage modern Java features (Java 21+)

Deliverables

1. Spring Boot source code + .env.example configuration.
2. Database migration script (Flyway/Liquibase).
3. API documentation (Swagger/OpenAPI).
4. Unit and integration tests.
5. README.md with project setup and run instructions.
6. (Optional) Dockerfile and docker-compose.yml.

Submission Instructions

- Create a GitHub/GitLab repository named: payment-gateway-cip.
- Use clear and structured commits (preferably conventional commits).
- Submit the repository URL to: irvan.wahyudi@collega.co.id

Sequence Flow Diagram

The following diagram illustrates the payment gateway transaction flow:

